



SISTEM REKOMENDASI

BAB 5

Knowledge based Recommender System

Ronny Susetyoko

D4 Sains Data Terapan

Pendahuluan



“Knowledge is knowing that a tomato is a fruit. Wisdom is knowing not to put it in a fruit salad.”–Brian O’Driscoll

Masalah Data Historis – Sistem berbasis konten dan kolaboratif membutuhkan banyak data historis, sehingga kurang efektif dalam kondisi cold-start.

Keterbatasan Sistem Kolaboratif & Konten – Sistem kolaboratif kesulitan menangani pengguna dan item baru, sementara sistem berbasis konten lebih baik untuk item baru tetapi tetap tidak bisa merekomendasikan bagi pengguna baru.

Ketidakefektifan pada Produk Khusus – Metode tradisional kurang efektif untuk produk kompleks seperti real estat, mobil, dan layanan keuangan karena minimnya data penilaian serta kompleksitas deskripsi produk.

Pendekatan Berbasis Pengetahuan – Sistem berbasis pengetahuan mengatasi masalah ini dengan mengumpulkan kebutuhan pengguna secara eksplisit dan menggunakan umpan balik interaktif untuk membantu eksplorasi pilihan.

Keunggulan untuk Pembelian Jarang – Sistem ini sangat cocok untuk keputusan pembelian yang jarang dilakukan dan membutuhkan informasi rinci sebelum pengguna membuat keputusan.

Pendahuluan



- ❖ Sistem rekomendasi berbasis pengetahuan **cocok ketika pengguna ingin menentukan kebutuhan secara eksplisit**, sulit mendapatkan penilaian karena kompleksitas produk, atau ketika peringkat produk berubah seiring waktu.
- ❖ **Keunggulan utama** sistem ini adalah kontrol pengguna dalam proses rekomendasi, berbeda dengan sistem berbasis konten dan kolaboratif yang bergantung pada data historis.
- ❖ Sistem ini sangat disesuaikan dengan domain tertentu melalui basis pengetahuan yang mencakup aturan dan metrik kesamaan.
- ❖ Beberapa juga mempertimbangkan atribut pengguna untuk meningkatkan relevansi rekomendasi.

Jenis Knowledge based RS



Constraint-based recommender systems

- Pengguna menentukan persyaratan atau batasan pada atribut item, lalu aturan domain digunakan untuk mencocokkan kebutuhan pengguna dengan atribut item.
- Jika hasil terlalu sedikit atau terlalu banyak, pengguna dapat menyesuaikan batasan secara interaktif.

Case-based recommender systems

- Pengguna memilih contoh item sebagai referensi, lalu sistem mencari item serupa berdasarkan metrik kesamaan yang spesifik untuk domain tersebut.
- Pengguna dapat menyempurnakan hasil dengan menyesuaikan atribut atau memberikan umpan balik untuk mempersempit pencarian.

Jenis Knowledge based RS

- Sistem rekomendasi berbasis pengetahuan memungkinkan pengguna mengubah kriteria pencarian, tetapi dengan cara berbeda:
 - **Case-based systems** menggunakan contoh sebagai titik acuan dengan metrik kesamaan untuk mencari item serupa.
 - **Constraint-based systems** menggunakan aturan atau batasan tertentu untuk membimbing pencarian.
 - Kedua metode ini memanfaatkan pengetahuan domain dalam bentuk aturan, batasan, atau metrik kesamaan. Karena itu, sistem berbasis pengetahuan sangat spesifik untuk tiap domain dan sulit diaplikasikan secara umum, meskipun prinsip dasarnya tetap sama di berbagai bidang.

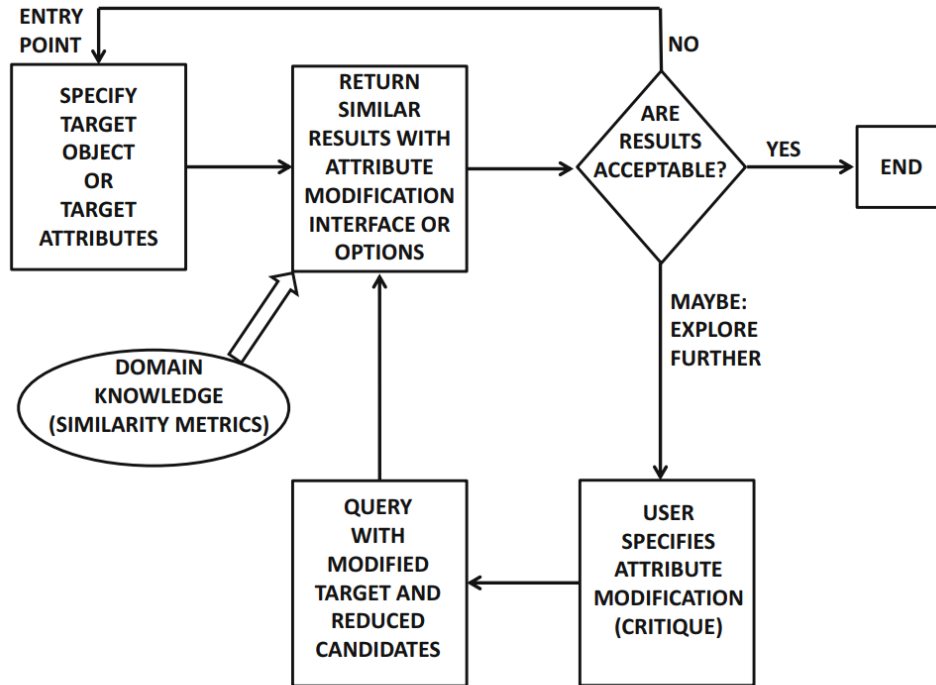


Bentuk Interaksi Pengguna Knowledge based RS

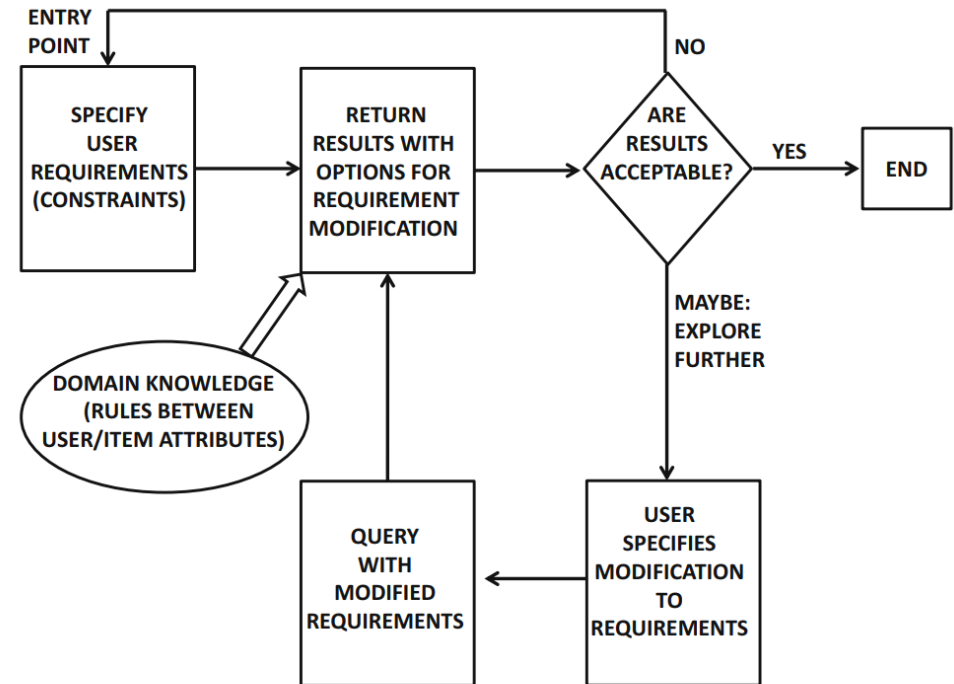


- **Conversational Systems** – Preferensi pengguna ditentukan melalui umpan balik iteratif dalam percakapan karena kompleksitas produk.
 - **Search-based Systems** – Preferensi pengguna dikumpulkan melalui serangkaian pertanyaan terstruktur, misalnya memilih antara rumah di kota atau pinggiran.
 - **Navigation-based Systems** – Pengguna meminta perubahan bertahap pada rekomendasi, seperti "*Saya ingin rumah serupa tetapi lebih ke barat 5 mil.*"
- Setiap metode cocok untuk sistem rekomendasi yang berbeda. **Critiquing systems** sering digunakan dalam case-based recommenders, sementara **search-based systems** lebih cocok untuk constraint-based recommenders. Kombinasi metode juga umum digunakan untuk membantu pengguna menavigasi pilihan dalam domain produk yang kompleks.

Gambaran Umum Proses Interaktif Knowledge based RS



(b) Case-based interaction



(a) Constraint-based interaction



contoh tabel atribut dalam aplikasi rekomendasi untuk pembelian rumah

Item-Id	Beds.	Baths.	Locality	Type	Floor Area	Price
1	3	2	Bronx	Townhouse	1600	220,000
2	5	2.5	Chappaqua	Split-level	3600	973,000
3	4	2	Yorktown	Ranch	2600	630,000
4	2	1.5	Yorktown	Condo	1500	220,000
5	4	2	Ossining	Colonial	2700	430,000

- Tabel ini dapat digunakan dalam sistem rekomendasi berbasis pengetahuan untuk mencocokkan preferensi pengguna dengan rumah yang tersedia.
- Tabel ini merepresentasikan atribut-atribut rumah yang dapat digunakan dalam sistem rekomendasi berbasis pengetahuan.

A. Constraint-Based Recommender Systems



- **Pencocokan Kendala:** Sistem rekomendasi kendala mencocokkan produk dengan persyaratan yang ditetapkan pengguna menggunakan aturan tertentu.
- **Perbedaan Atribut:** Pengguna sering menyatakan kebutuhan dengan atribut berbeda dari atribut produk, sehingga diperlukan pemetaan tambahan.
- **Contoh Properti:** Dalam pembelian rumah, ukuran keluarga dan preferensi lokasi dipetakan ke jumlah kamar tidur dan lokasi properti.
- **Contoh Keuangan:** Dalam investasi, preferensi konservatif diterjemahkan menjadi aset seperti Treasury Bonds.
- **Basis Pengetahuan:** Aturan tambahan dalam basis pengetahuan membantu sistem menyaring dan merekomendasikan opsi yang sesuai.



Constraint-Based Recommender Systems

- Filter conditions memetakan kebutuhan pengguna ke atribut produk untuk menyaring hasil rekomendasi, biasanya berdasarkan aturan domain atau analisis historis.
- Dalam otomotif, misalnya, fitur tertentu hanya tersedia untuk model spesifik (compatibility conditions), yang membantu mendeteksi ketidakkonsistenan dalam preferensi pengguna.
- Beberapa compatibility constraints juga menghubungkan atribut pelanggan, seperti aturan bahwa individu lajang cenderung memilih rumah lebih kecil (**Marital-status=single $\Rightarrow$ Min-Bedrooms $\leq$ 5**) atau keluarga besar membutuhkan lebih banyak kamar (**Family-Size $\geq$ 5 $\Rightarrow$ Min-Bedrooms $\geq$ 3**).



Constraint-Based Recommender Systems

- Atribut Pengguna: Sistem menerima input berupa demografi dan preferensi produk, ditentukan dalam satu sesi tanpa personalisasi lintas sesi.
- Basis Pengetahuan: Memetakan atribut pengguna ke atribut produk menggunakan aturan tertentu.
- Aturan Langsung: Menghubungkan kebutuhan pengguna ke atribut produk, misalnya **Min-Bedrooms $\geq 3 \Rightarrow$ Price $\geq 100,000$** .
- Aturan Tidak Langsung: Mengaitkan atribut pengguna dengan ekspektasi produk, seperti **Family-Size $\geq 5 \Rightarrow$ Min-Bedrooms ≥ 3** .
- Konsistensi Preferensi: Jika aturan bertentangan dengan preferensi pengguna, sistem tidak akan merekomendasikan item apa pun.



Constraint-Based Recommender Systems

- Basis pengetahuan dalam sistem rekomendasi berbasis kendala berasal dari informasi publik, pakar domain, pengalaman sebelumnya, atau data mining dari histori data, sehingga memerlukan usaha besar dalam pembangunannya.
- Selain itu, katalog produk berisi daftar produk beserta atributnya, seperti yang ditunjukkan dalam Tabel 5.2. Dengan demikian, tantangan utama adalah menemukan produk dalam katalog yang memenuhi kebutuhan pelanggan serta aturan dalam basis pengetahuan.



Pengembalian Hasil Yang Relevan

- Masalah pencarian hasil relevan dapat direduksi menjadi penyaringan data pada katalog. Persyaratan pelanggan dan aturan yang berlaku digunakan untuk membentuk kueri seleksi database melalui langkah berikut:
 - **Identifikasi aturan yang berlaku:** Atribut pelanggan diperiksa terhadap aturan dalam basis pengetahuan. Jika cocok, aturan ditambahkan ke persyaratan pelanggan hingga tidak ada aturan baru yang berlaku.
 - **Penyusunan kueri seleksi:** Persyaratan pelanggan diterjemahkan menjadi kueri database dalam bentuk normal konjungtif, mencerminkan batasan pada katalog produk.
 - **Pengambilan hasil:** Kueri digunakan untuk mengambil produk yang sesuai.
- Sistem ini umumnya tidak menyimpan data pengguna, sehingga hasil yang diberikan bersifat statis. Pengguna dapat menyempurnakan pencariannya untuk mendapatkan rekomendasi yang lebih sesuai.



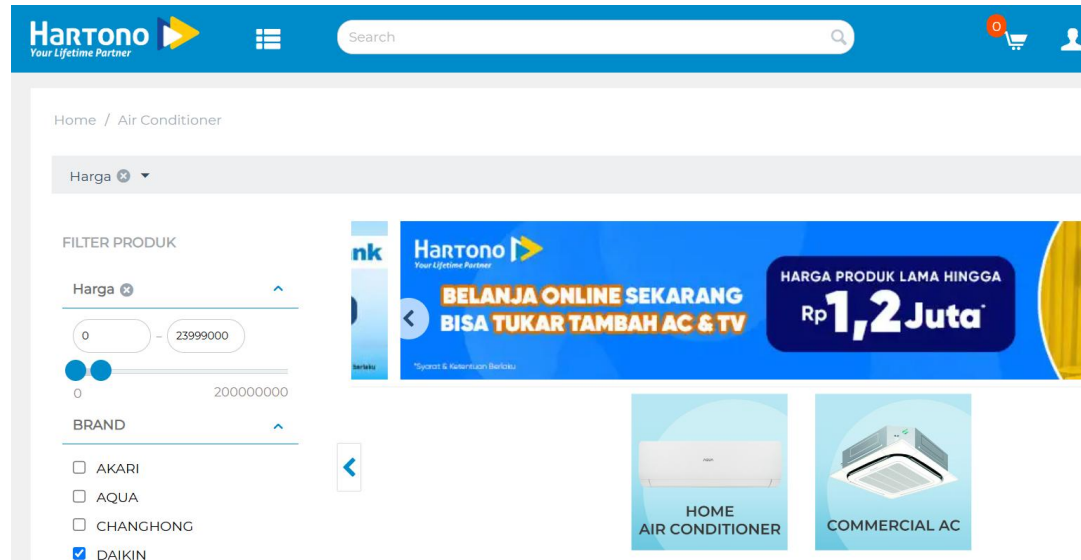
Pendekatan Interaksi

- **Input Preferensi Awal:** Pengguna memasukkan preferensi awal melalui antarmuka interaktif, seperti formulir web atau serangkaian pertanyaan.
- **Penyajian Hasil:** Sistem menampilkan daftar hasil yang diberi peringkat dan menjelaskan alasannya. Jika tidak ada hasil yang cocok, sistem menyarankan pelonggaran batasan. Jika terlalu banyak hasil, sistem menyarankan penambahan batasan.
- **Penyempurnaan Kriteria:** Pengguna dapat menyesuaikan kriteria berdasarkan hasil yang diterima, baik dengan menambahkan atau menghapus batasan. Sistem membantu dengan metode pemenuhan batasan untuk memberikan saran penyempurnaan.

Sistem juga menangani atribut yang tidak ditentukan pengguna dengan dua cara:

- Membiarkan atribut tidak dibatasi dan menampilkan semua kemungkinan.
- Menyediakan nilai default berdasarkan aturan domain atau data historis pengguna.

Setelah kueri dibuat, sistem menyusun daftar hasil yang relevan, memberikan peringkat, serta memberikan penjelasan dan panduan untuk penyempurnaan kueri.



EXAMPLE OF HYPOTHETICAL CONSTRAINT-BASED INTERFACES FOR HOME BUYING (constraint-example.com)

[ENTRY POINT]

I WOULD LIKE TO BUY A HOUSE SATISFYING THE FOLLOWING REQUIREMENTS:

MIN. BR	MAX. BR	MIN. BATH	MAX. BATH
MIN. PRICE	MAX. PRICE	HOME STYLE	ZIP CODE

SUBMIT SEARCH

- Contoh hipotetis dari antarmuka pengguna awal untuk rekomendasi berbasis kendala (constraint-example.com)

contoh hipotetis dari antarmuka pengguna untuk menangani hasil kueri kosong dalam rekomendasi berbasis kendala (constraint-example.com)



EXAMPLE OF HYPOTHETICAL CONSTRAINT-BASED INTERFACE FOR HOME BUYING (constraint-example.com)

[CONSTRAINT MODIFICATION INTERFACE]

YOU SPECIFIED THE FOLLOWING REQUIREMENTS (CURRENT VALUES IN BRACKETS):

MIN. BR (5)	MAX. BR	MIN. BATH	MAX. BATH (1)
MIN. PRICE	MAX. PRICE (\$70K)	HOME STYLE	ZIP CODE (10547)

SUBMIT MODIFICATION **GO BACK TO ENTRY POINT**

YOUR QUERY RETURNED 0 RESULTS. MODIFY YOUR SEARCH ACCORDING TO THE SUGGESTIONS BELOW:

- EITHER REDUCE MIN. BR FROM 5 OR INCREASE MAX. PRICE FROM \$70K.
- EITHER REDUCE MIN. BR FROM 5 OR INCREASE MAX. BATH FROM 1.
- EITHER INCREASE MAX. PRICE OR CHANGE ZIP CODE FROM 10547.

MORE DETAILS

- Antarmuka ini memberi tahu pengguna bahwa kueri mereka tidak menghasilkan hasil (0 results) dan memberikan beberapa saran untuk memodifikasi pencarian agar sesuai dengan batasan yang lebih longgar.
- Contoh:
 - Mengurangi jumlah minimum kamar tidur
 - Meningkatkan harga maksimum
 - Mengubah kode pos

Peringkat Item yang Cocok



Sistem dapat memberi peringkat hasil rekomendasi dengan beberapa metode:

❖ Peringkat Berdasarkan Satu Atribut

- Pengguna memilih satu atribut numerik sebagai dasar peringkat, seperti harga rumah atau jumlah kamar tidur.
- Metode ini sederhana tetapi mengabaikan atribut lain yang mungkin penting.

❖ Peringkat Menggunakan Fungsi Utilitas

- Setiap item direpresentasikan sebagai vektor atribut $V = (v_1, v_2, \dots, v_d)$.
- Atribut memiliki bobot w_j dan fungsi kontribusi $f_j(v_j)$.
- Nilai utilitas keseluruhan $U(V)$ dihitung sebagai kombinasi bobot dan kontribusi setiap atribut.

Metode berbasis fungsi utilitas lebih fleksibel dan memungkinkan pengambilan keputusan yang lebih optimal dibandingkan dengan pendekatan berbasis satu atribut.

$$U(\bar{V}) = \sum_{j=1}^d w_j \cdot f_j(v_j)$$



Menangani Hasil yang Tidak Dapat Diterima atau Set Kosong

- Ketika kueri tidak menghasilkan hasil yang cukup, pengguna memiliki dua pilihan: memulai ulang atau melonggarkan batasan. Agar pengguna dapat mengambil keputusan yang tepat, sistem dapat memberikan repair proposals, yaitu saran pelonggaran batasan dengan mengidentifikasi minimal set of inconsistent constraints.
- Contohnya, jika pengguna menginginkan $\text{Max-Price} < 100,000$ dan $\text{Min-Bedrooms} > 5$, tetapi tidak ada hasil yang sesuai, sistem dapat menunjukkan bahwa salah satu dari batasan ini harus diubah.
- Metode dasar untuk menemukan batasan yang tidak konsisten adalah pencarian bottom-up, tetapi ini bisa tidak efisien jika jumlah batasan besar.
- Alternatifnya, algoritma seperti QUICKXPLAIN dan MINRELAX dapat menemukan konflik lebih cepat. Secara praktis, sistem sebaiknya menyajikan kelompok kecil batasan yang tidak konsisten, sehingga pengguna dapat memahami konflik dan menyesuaikan preferensinya dengan lebih intuitif.



Penambahan Batasan

- Jika hasil pencarian terlalu banyak, sistem menyarankan batasan tambahan berdasarkan **analisis log sesi historis**.
 - **Pola Populer** – Sistem mencari atribut tambahan yang sering digunakan dalam sesi sebelumnya dengan batasan serupa.
 - **Urutan Temporal** – Menggunakan *sequential pattern mining* dan *MDP* untuk menyarankan atribut berdasarkan pola urutan pengguna sebelumnya.
 - **Selektivitas** – Menentukan batasan yang paling efektif dalam menyaring hasil.



Contoh Constraint-Based Recommender Systems

- Seorang pelanggan ingin membeli laptop dengan spesifikasi tertentu. Sistem rekomendasi berbasis constraint akan menyaring produk berdasarkan kriteria yang diberikan pelanggan.
- Kriteria Pelanggan:
 - Harga maksimal: Rp15.000.000
 - RAM minimal: 8GB
 - Penyimpanan minimal: 512GB SSD
 - Ukuran layar: Minimal 14 inci
 - Merek yang diinginkan: Dell, ASUS, atau Lenovo

Contoh Constraint-Based Recommender Systems



```
# Daftar laptop tersedia di database
```

```
laptops = [  
    {"brand": "Dell", "price": 14000000, "ram": 16, "storage": 512, "screen_size": 14},  
    {"brand": "ASUS", "price": 12000000, "ram": 8, "storage": 512, "screen_size": 15.6},  
    {"brand": "Lenovo", "price": 16000000, "ram": 16, "storage": 1024, "screen_size": 14},  
    {"brand": "Acer", "price": 11000000, "ram": 8, "storage": 256, "screen_size": 14},  
    {"brand": "HP", "price": 15000000, "ram": 16, "storage": 512, "screen_size": 13.3},  
    {"brand": "ASUS", "price": 9000000, "ram": 8, "storage": 512, "screen_size": 14},  
    {"brand": "Dell", "price": 17000000, "ram": 16, "storage": 1024, "screen_size": 15.6}  
]
```

```
# Constraint pengguna
```

```
max_price = 15000000
```

```
min_ram = 8
```

```
min_storage = 512
```

```
min_screen_size = 14
```

```
preferred_brands = {"Dell", "ASUS", "Lenovo"}
```

Contoh Constraint-Based Recommender Systems



```
# Filter laptop berdasarkan constraint
recommended_laptops = [
    laptop for laptop in laptops
    if (laptop["price"] <= max_price and
        laptop["ram"] >= min_ram and
        laptop["storage"] >= min_storage and
        laptop["screen_size"] >= min_screen_size and
        laptop["brand"] in preferred_brands)
]
# Menampilkan rekomendasi
if recommended_laptops:
    print("Laptop yang direkomendasikan:")
    for laptop in recommended_laptops:
        print(f"{laptop['brand']} - Rp{laptop['price']} - RAM {laptop['ram']}GB - {laptop['storage']}GB SSD - {laptop['screen_size']} inci")
else:
    print("Tidak ada laptop yang sesuai dengan kriteria.")
```

Laptop yang direkomendasikan:

Dell - Rp14000000 - RAM 16GB - 512GB SSD - 14 inci
ASUS - Rp12000000 - RAM 8GB - 512GB SSD - 15.6 inci
ASUS - Rp9000000 - RAM 8GB - 512GB SSD - 14 inci



B. Case-Based Recommenders

- **Pendekatan Berbasis Kemiripan**

- Menggunakan metrik kesamaan untuk menemukan item yang paling mirip dengan preferensi pengguna.
- Tidak ada batasan keras seperti pada sistem berbasis aturan. Jika tidak ada hasil yang cocok, sistem tetap menampilkan item dengan kemiripan tertinggi.

- **Metode Refinement (Penyempurnaan)**

- Critiquing: Pengguna bisa memilih hasil dan meminta variasi tertentu, misalnya: "Berikan saya lebih banyak rumah seperti ini, tetapi dengan lebih banyak kamar tidur."
- Membantu pengguna menjelajahi opsi dengan cara interaktif.

- **Proses Iteratif**

- Hasil pencarian menyempit di setiap iterasi, berbeda dengan sistem berbasis aturan yang mungkin menambahkan hasil baru.
- Dalam beberapa desain, pencarian bisa diperluas agar pengguna bisa menjelajahi opsi lebih luas, tetapi berisiko menghasilkan hasil yang kurang relevan.



Case-Based Recommenders

Agar sistem rekomendasi berbasis kasus dapat berfungsi secara efektif, terdapat dua aspek penting yang harus dirancang dengan baik:

- **Metrik Kemiripan:**

Perancangan metrik kemiripan yang efektif sangat penting dalam sistem berbasis kasus untuk mendapatkan hasil yang relevan. Setiap atribut harus diperhitungkan dengan tepat dalam fungsi kemiripan agar sistem dapat bekerja secara optimal.

- **Metode Critiquing:**

Eksplorasi interaktif terhadap ruang item didukung melalui penggunaan metode critiquing. Berbagai metode critiquing tersedia untuk mendukung tujuan eksplorasi yang berbeda-beda.

Metrik Similarity



Perancangan metrik kemiripan yang tepat sangat penting dalam sistem rekomendasi berbasis kasus. Sistem awal seperti FindMe menyusun atribut berdasarkan tingkat kepentingan dan melakukan penyortiran bertahap, tetapi pendekatan ini tidak selalu efektif. Oleh karena itu, lebih baik menggunakan fungsi kemiripan yang dapat disesuaikan oleh pakar atau dipelajari secara otomatis. Dalam sistem dengan d atribut, kemiripan antara dua vektor parsial X dan T dihitung menggunakan fungsi tertentu.

$$Sim(t_i, x_i) = 1 - \frac{|t_i - x_i|}{max_i - min_i}$$

Bobot w_i menentukan kepentingan relatif setiap atribut.

Atribut dapat bersifat **simetris** (nilai target harus mendekati pengguna) atau **asimetris** (nilai lebih besar atau lebih kecil lebih disukai). Untuk atribut asimetris, fungsi kemiripan dapat mencakup *reward*:

$$Sim(t_i, x_i) = 1 - \frac{|t_i - x_i|}{max_i - min_i} + \alpha_i \cdot I(x_i > t_i) \cdot \frac{|t_i - x_i|}{max_i - min_i}$$

Parameter α_i menentukan seberapa besar preferensi pengguna terhadap nilai yang lebih tinggi atau lebih rendah.

Metrik Similarity



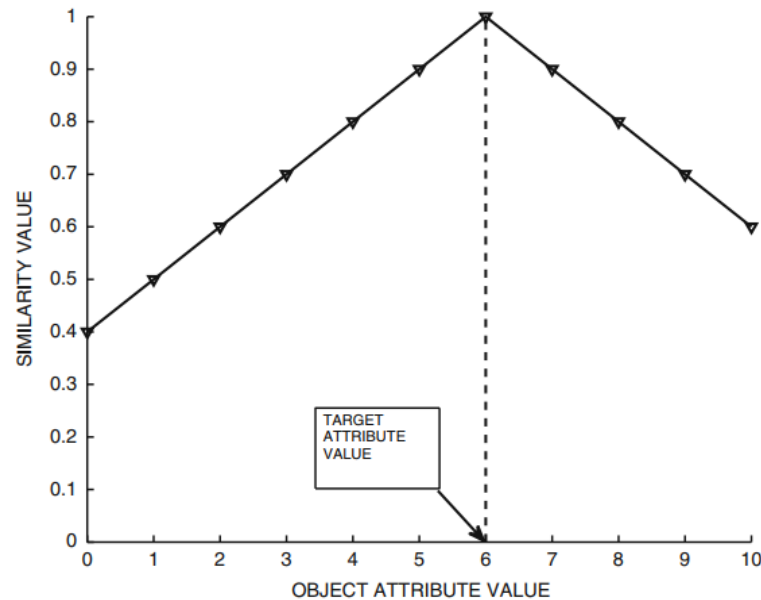
Fungsi $\text{Sim}(t_i, x_i)$ lebih tepat dianggap sebagai fungsi utilitas daripada kemiripan dalam beberapa kasus.

- $\alpha_i = 1.0$: Tidak peduli perubahan di satu arah (misal, resolusi kamera di atas batas tertentu).
- $\alpha_i \in (0, 1)$: Preferensi asimetris terhadap nilai target (misal, tenaga kuda lebih tinggi disukai, tapi konsumsi bahan bakar berlebihan tidak).

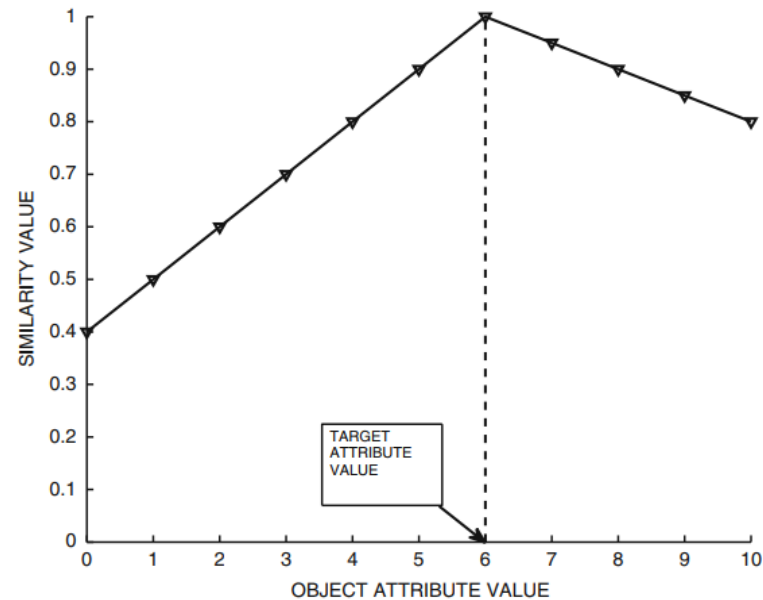
Jenis fungsi kemiripan:

- **Simetris**: Linear dari nilai target.
- **Asimetris**: Preferensi berbeda di atas/bawah target (misal, harga lebih rendah lebih baik).
- **Batas utilitas**: Kemiripan tetap setelah titik tertentu (misal, resolusi kamera).

Untuk data kategorikal, kemiripan sering berbasis hierarki domain atau pembelajaran dari umpan balik pengguna.



(a) Symmetric ($\alpha_i = 0$)
(penalty by absolute distance)

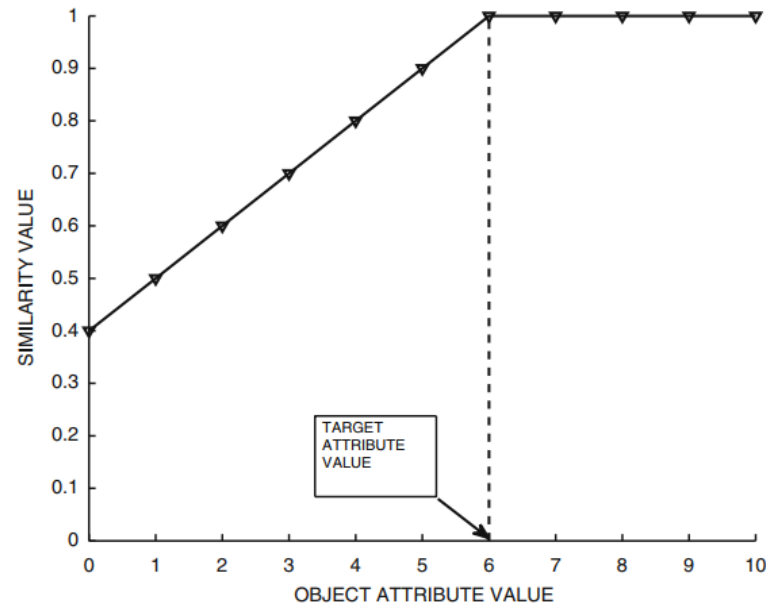


(b) Asymmetric ($\alpha_i = 0.5$)
(milder penalty for overshooting)

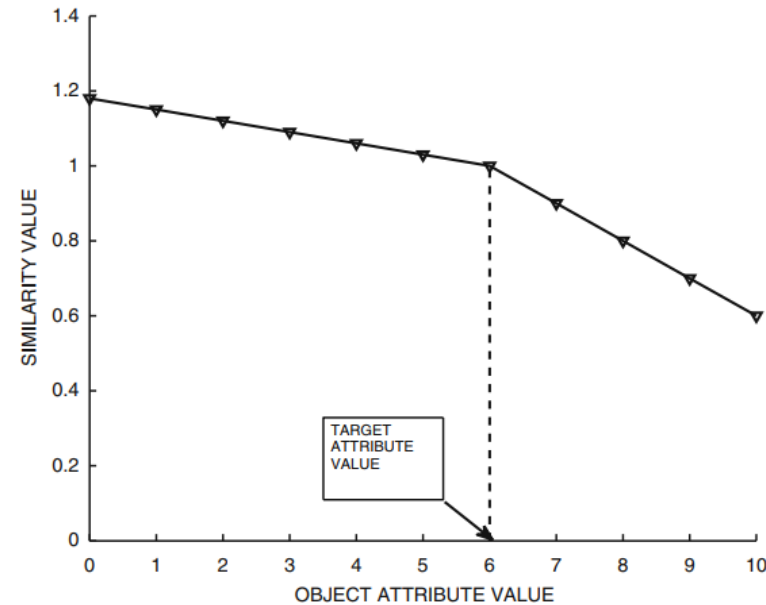
Dua metode perhitungan kemiripan dalam sistem rekomendasi:

- **Simetris ($\alpha_i = 0$)** → Kemiripan menurun secara seimbang saat atribut menjauh dari target.
- **Asimetris ($\alpha_i = 0.5$)** → Penalti lebih ringan jika atribut melebihi target dibanding jika kurang dari target.

Metode ini digunakan untuk mencerminkan preferensi pengguna terhadap suatu atribut.



(a) Asymmetric ($\alpha_i = 1.0$)
(no penalty for overshooting)



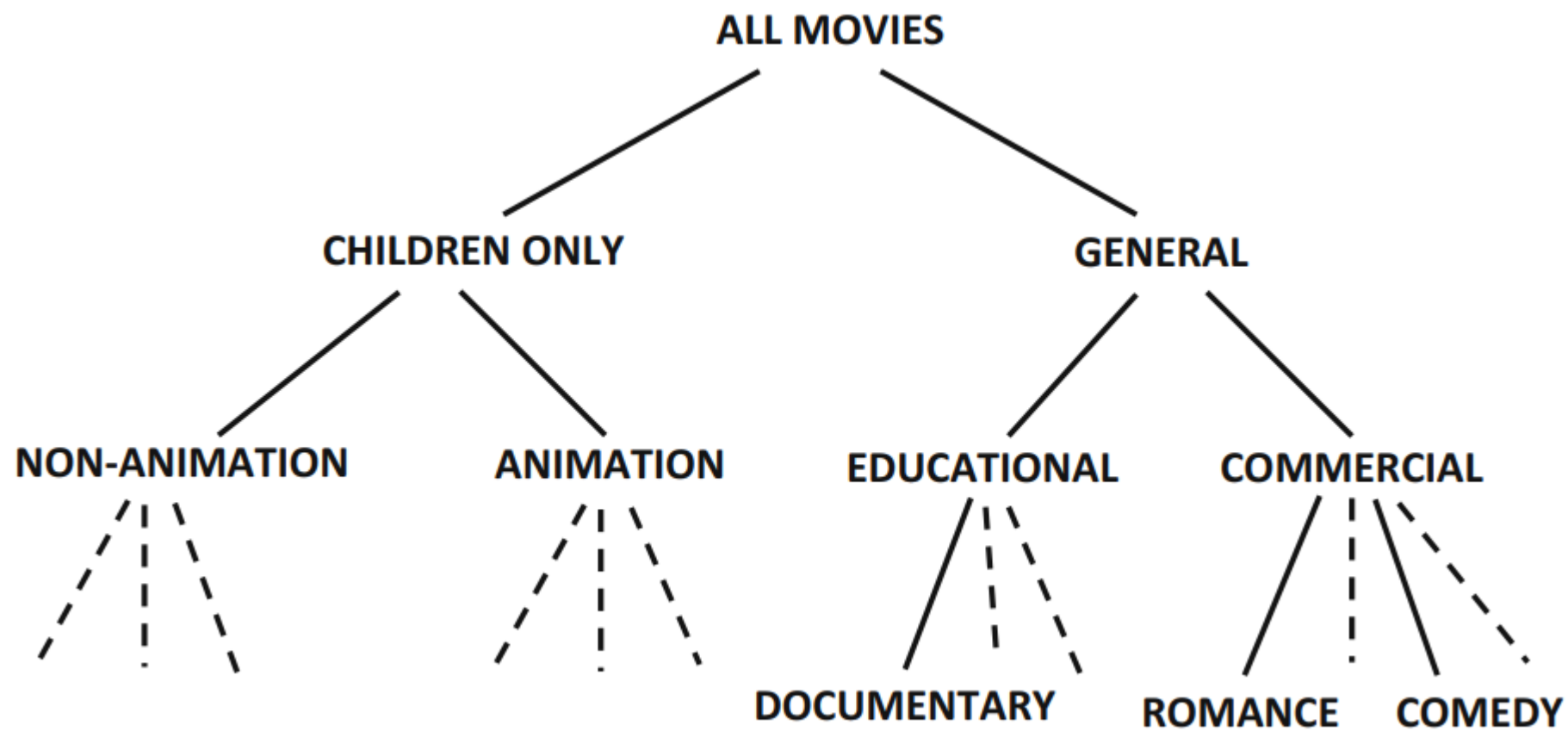
(b) Asymmetric ($\alpha_i = 1.3$)
(less is always better)

Dua bentuk fungsi kemiripan asimetris:

(a) Asimetris ($\alpha_i = 1.0$) → Tidak ada penalti untuk nilai yang melebihi target, artinya semakin besar nilainya tetap dianggap baik.

(b) Asimetris ($\alpha_i = 1.3$) → Semakin kecil lebih baik, sehingga kemiripan menurun saat atribut bertambah.

Fungsi ini digunakan untuk menangkap preferensi pengguna dalam sistem rekomendasi.





- Kritik dalam sistem rekomendasi memungkinkan pengguna menyempurnakan permintaan mereka setelah melihat hasil pencarian. Karena sering kali sulit bagi pengguna untuk menyatakan kebutuhan mereka secara akurat sejak awal, kritik memungkinkan mereka mengajukan perubahan pada atribut tertentu.
- Pengguna dapat memberikan kritik **arah** (misalnya, "lebih murah") atau **penggantian** (misalnya, "warna berbeda"). Sistem kemudian menyaring hasil yang tidak sesuai dan menampilkan alternatif yang lebih relevan.
- Ada tiga jenis kritik utama:
 - **Kritik sederhana** – perubahan pada satu fitur.
 - **Kritik gabungan** – perubahan pada beberapa fitur sekaligus.
 - **Kritik dinamis** – perubahan berdasarkan interaksi sebelumnya, dengan prioritas pada kritik terbaru.

Kritik Sederhana



Dalam sistem rekomendasi, seperti **FindMe**.

Keunggulan:

- Memudahkan pengguna mengubah satu fitur tanpa perlu memahami atribut produk secara mendalam.
- Memungkinkan kritik arah seperti "lebih murah" atau "lebih kuat" tanpa menentukan nilai spesifik.

Kelemahan:

- Navigasi bisa melelahkan jika banyak fitur yang perlu diubah.
- Perubahan satu fitur bisa mempengaruhi atribut lain berdasarkan ketersediaan item.
- Pengguna mungkin tidak memahami trade-off antar fitur (misalnya, tenaga mesin vs efisiensi bahan bakar).
- Bisa terjebak dalam siklus buntu jika hanya mempertimbangkan perubahan terbaru dan mengabaikan opsi sebelumnya.

Kritik Sederhana



**EXAMPLE OF HYPOTHETICAL CASE-BASED RECOMMENDATION
INTERFACE FOR HOME BUYING (critique-example.com)**

[SIMPLE CRITIQUING INTERFACE]

YOU SPECIFIED THE FOLLOWING TARGET:
812 SCENIC DRIVE, MOHEGAN LAKE, NY

YOUR TOP RECOMMENDATION IS:
742 SCENIC DRIVE, MOHEGAN LAKE, NY

WE RECOMMEND THIS HOUSE BECAUSE: IT HAS SIMILAR BEDROOMS, BATHROOMS,
LOCALITY, PRICE RANGE, AND HOME STYLE AS YOUR TARGET

**I WOULD LIKE TO BUY A HOUSE SIMILAR TO THE TOP RECOMMENDATION
BUT WITH **ONE** OF THE FOLLOWING CHANGES:**

NUMBER OF BR ▼	SUBMIT CHANGE	NUMBER OF BATH ▼	SUBMIT CHANGE
PRICE RANGE ▼	SUBMIT CHANGE	ZIP CODE	SUBMIT CHANGE
HOME STYLE ▼	SUBMIT CHANGE		

SEE OTHER RESULTS GO BACK TO ENTRY POINT

Kritik sederhana memungkinkan pengguna mengubah satu fitur dari rekomendasi tanpa mengulang pencarian. Pengguna memilih atribut (misalnya, harga atau jumlah kamar) lalu mengklik "**Submit Change**" untuk mendapat rekomendasi baru.

Keunggulan: Mudah digunakan, tidak perlu memasukkan banyak info ulang.

Kelemahan: Bisa melelahkan jika banyak perubahan diperlukan, berisiko terjebak dalam siklus buntu.



- Kritik Gabungan dikembangkan untuk mempercepat proses rekomendasi dengan memungkinkan pengguna mengubah beberapa fitur sekaligus dalam satu siklus. Misalnya, sistem *Car Navigator* menggunakan deskripsi intuitif seperti "lebih mewah" atau "lebih luas," yang secara otomatis menerjemahkan ke beberapa perubahan fitur.
- Keuntungan:
 - - Memungkinkan perubahan banyak fitur dalam satu langkah, mengurangi jumlah siklus rekomendasi.
 - - Mempermudah eksplorasi produk dengan lompatan besar dalam ruang fitur.
 - - Memberikan kontrol lebih besar kepada pengguna.
- Kelemahan:
 - - Tidak selalu membantu pengguna memahami trade-off antara fitur produk.
 - - Memerlukan usaha besar dari pakar domain untuk merancang sistem dan mengkodekan perubahan fitur.
 - - Siklus yang lebih singkat dapat mengurangi kesempatan pengguna untuk belajar lebih dalam tentang hubungan antar fitur.



Kritik Dinamis bertujuan meningkatkan efisiensi eksplorasi dengan menyesuaikan opsi kritik berdasarkan hasil yang telah diambil, berbeda dengan kritik gabungan yang statis. Dengan menggunakan data mining, sistem menemukan kombinasi perubahan fitur yang paling relevan, memandu pengguna secara lebih efektif.

- Keunggulan:
 - - Menyesuaikan opsi berdasarkan hasil yang sudah ditemukan, menghindari pencarian sia-sia.
 - - Menggunakan pola frekuensi dari hasil yang diperoleh untuk menyusun kritik yang lebih relevan.
 - - Mengurangi beban kognitif secara keseluruhan dengan mempercepat pencarian hasil yang sesuai.
- Kelemahan:
 - - Meningkatkan beban kognitif dalam satu siklus karena pilihan lebih kompleks.
 - - Membutuhkan mekanisme penjelasan yang baik agar pengguna memahami rekomendasi dengan jelas.

Contoh Case-Based Recommenders



Sebuah toko elektronik ingin mengembangkan sistem rekomendasi laptop berbasis **case-based reasoning**.

- ❖ Ketika seorang pelanggan mencari laptop, sistem akan mencari produk yang mirip dengan preferensi pelanggan berdasarkan spesifikasi yang diberikan (misalnya, RAM, penyimpanan, dan harga).
- ❖ Jika tidak ada laptop yang persis sesuai, sistem akan merekomendasikan produk yang paling mirip berdasarkan kesamaan fitur.

Program ini mencari laptop dengan spesifikasi paling mirip dengan preferensi pengguna menggunakan jarak Euclidean.

Jika pengguna menginginkan **16GB RAM, 512GB storage, dan harga sekitar \$1000**, sistem akan merekomendasikan **Laptop B** atau **Laptop C**, tergantung mana yang lebih dekat secara fitur.

Contoh Case-Based Recommenders



```
import numpy as np
from sklearn.metrics.pairwise import euclidean_distances
# Data contoh: daftar laptop dengan spesifikasi (RAM, Storage, Price)
laptops = [
    {"name": "Laptop A", "ram": 8, "storage": 256, "price": 700},
    {"name": "Laptop B", "ram": 16, "storage": 512, "price": 1200},
    {"name": "Laptop C", "ram": 8, "storage": 512, "price": 900},
    {"name": "Laptop D", "ram": 16, "storage": 1024, "price": 1500},
    {"name": "Laptop E", "ram": 32, "storage": 1024, "price": 2000},
]
# Konversi data ke dalam array numerik
features = np.array([[l["ram"], l["storage"], l["price"]] for l in laptops])
def recommend_laptop(user_pref):
    """ Mencari laptop yang paling mirip dengan preferensi pengguna. """
    user_features = np.array(user_pref).reshape(1, -1)
    distances = euclidean_distances(features, user_features)
    best_match_idx = np.argmin(distances)
    return laptops[best_match_idx]
# Input preferensi pengguna (RAM, Storage, Price)
user_pref = [16, 512, 1000] # Contoh pengguna mencari laptop 16GB RAM, 512GB Storage, harga 1000$
recommended = recommend_laptop(user_pref)
print(f"Rekomendasi untuk Anda: {recommended['name']} (RAM: {recommended['ram']}GB, Storage: {recommended['storage']}GB, Price: ${recommended['price']})")
```

Rekomendasi untuk Anda: Laptop C (RAM: 8GB, Storage: 512GB, Price: \$900)

Personalisasi Berkelanjutan dalam Knowledge-Based Systems



Sistem berbasis kendala memungkinkan pengguna menentukan preferensi dalam satu sesi, tetapi tidak menyimpan data permanen. Ini berbeda dari sistem berbasis konten dan kolaboratif yang menggunakan data historis untuk memberikan rekomendasi.

Keunggulan:

- -Tidak mengalami masalah *cold-start* seperti sistem berbasis histori.
- Cocok untuk produk yang jarang dibeli dan sangat disesuaikan.

Personalisasi dalam Sistem Berbasis Pengetahuan:

1. Fungsi Utilitas/Similitas: Pembelajaran terhadap atribut yang lebih penting bagi pengguna berdasarkan umpan balik sebelumnya.
2. Saran Kendala: Dapat dipersonalisasi jika cukup banyak data sesi pengguna tersedia.
3. Kritik Dinamis: Disesuaikan berdasarkan pola yang relevan dengan pengguna, atau diperluas dengan data pengguna lain yang memiliki preferensi serupa.

Tantangan:

- Keterbatasan data sesi pengguna membuat personalisasi sulit diterapkan.
- Produk yang ditangani sering kali memiliki domain kompleks dan unik, sehingga sulit untuk menggeneralisasi preferensi pengguna.

Ringkasan



Sistem rekomendasi berbasis pengetahuan cocok untuk domain dengan item yang sangat disesuaikan, di mana preferensi tidak dapat dengan mudah direpresentasikan melalui peringkat. Sistem ini memberi pengguna lebih banyak kontrol dalam proses rekomendasi melalui spesifikasi kebutuhan dan interaktivitas.

Jenis Sistem Berbasis Pengetahuan:

1. Sistem Berbasis Kendala:

- Pengguna menentukan persyaratan yang dikombinasikan dengan aturan domain.
- Kendala dapat ditambah atau dikurangi sesuai kebutuhan hasil pencarian.

2. Sistem Berbasis Kasus:

- Menggunakan daftar kandidat yang dimodifikasi secara iteratif melalui kritik (*critiquing*).
- Retrieval menggunakan fungsi kesamaan spesifik domain yang bisa dipelajari.
- Modifikasi dilakukan dengan kritik sederhana, gabungan, atau dinamis.

Keunggulan: Efektif dalam menangani masalah *cold-start*.

Kelemahan: Tidak memanfaatkan data historis untuk menyempurnakan rekomendasi secara otomatis.

Terima Kasih

